JavaScript Recap Notes

Module 3: JavaScript Basics • Functions, Arrays & Objects

A handout for beginners — key concepts, plain-English explanations, and worked code examples.

Contents

Part 1 — JavaScript Basics

1. Getting Started with JavaScript

JavaScript (JS) is a programming language that runs in the browser (and, with Node.js, outside the browser too). It's what makes web pages interactive — reacting to clicks, updating content, validating forms, and more.

There are a few common ways to run JavaScript:

- **In the browser console** — open DevTools (F12 or right-click → Inspect), go to the **Console** tab, and type code directly.
- **Inside HTML** — using a `<script>` tag, either inline or linking to an external `.js` file.
- **With Node.js** — run a `.js` file from the terminal using `node filename.js`.

The most common tool for output while learning is `console.log()`, which prints a value to the console.

```
// index.html
<script src="app.js"></script>

// app.js
console.log("Hello, world!");

// A comment on its own line
/* A comment
   spanning multiple lines */
```

Key Points

- JS code is made up of **statements**, usually ending in a semicolon `;`.
- `console.log()` is your best friend for checking what your code is doing.
- Use `//` for a single-line comment and `/* */` for a multi-line comment.

2. Variables, Types & Operators

Variables store data so we can use and reuse it. JavaScript gives us three keywords to declare variables:

- `let` — can be reassigned later. Use this by default.
- `const` — cannot be reassigned. Use this when the value shouldn't change.
- `var` — the old way of declaring variables; avoid it in modern code (it behaves less predictably).

```
let score = 10;
score = 15;          // OK — let can change

const name = "Amy";
```

```
// name = "Bob";    // Error — const cannot be reassigned
```

Every value in JavaScript has a type. The basic (primitive) types are:

- **Number** — e.g. `5`, `3.14`, `-2`
- **String** — text, wrapped in quotes: `"hello"`, `'hello'`, or backticks for templates
- **Boolean** — `true` or `false`
- **Undefined** — a variable that has been declared but not given a value
- **Null** — an intentionally empty value

You can check a value's type with the `typeof` operator.

```
typeof 5;            // "number"
typeof "hi";          // "string"
typeof true;          // "boolean"
typeof undefined;     // "undefined"

// Template literals let you insert variables into a string
let age = 16;
console.log(`I am ${age} years old.`);
```

Operators let us work with values:

- **Arithmetic**: `+ - * / % **` (add, subtract, multiply, divide, remainder, power)
- **Assignment**: `= += -= *= /=`
- **Comparison**: `===` and `!==` check value AND type (use these); `==` and `!=` only check value and can give surprising results — avoid them.
- **Logical**: `&&` (and), `||` (or), `!` (not)

Key Points

- Prefer `let` and `const` over `var`.
- Use `const` unless you know the value needs to change.
- Always use `===` / `!==` instead of `==` / `!=`.

3. Control Flow

Control flow is how we make decisions and repeat actions in code.

Conditionals

```
let temperature = 30;

if (temperature > 25) {
```

```
  console.log("It's hot!");
} else if (temperature > 15) {
  console.log("It's mild.");
} else {
  console.log("It's cold!");
}
```

A `switch` statement is a tidy alternative when you're checking one value against several exact options:

```
switch (day) {
  case "Mon":
    console.log("Start of the week");
    break;
  case "Fri":
    console.log("Almost the weekend!");
    break;
  default:
    console.log("Just another day");
}
```

Every value is either "truthy" or "falsy" when used in a condition. The falsy values are: `false`, `0`, `""` (empty string), `null`, `undefined`, and `NaN`. Everything else is truthy.

Loops

- `for` — repeat a fixed number of times, using a counter.
- `while` — repeat while a condition is true (checked before each pass).
- `do...while` — like `while`, but always runs at least once.
- `for...of` — loop over the values in an array (covered more in Lesson 7).

```
for (let i = 0; i < 5; i++) {
  console.log(i);        // 0, 1, 2, 3, 4
}

let count = 0;
while (count < 3) {
  console.log(count);
  count++;
}
```

`break` exits a loop completely. `continue` skips to the next round of the loop.

Key Points

- `if / else if / else` handles different conditions in order.
- `for` is great when you know how many times to repeat; `while` is great when you don't.
- Remember to update your loop counter, or you'll create an **infinite loop**!

4. Scope & Debugging

Scope determines where in your code a variable can be accessed.

- **Global scope** — declared outside any function or block; accessible everywhere.
- **Function scope** — declared inside a function; only accessible inside that function.
- **Block scope** — `let` and `const` declared inside `{ }` (like an `if` or `for`) are only accessible inside that block. `var` ignores block scope, which is one reason to avoid it.

```
function greet() {
  let message = "Hi!";   // function-scoped
  if (true) {
    let localOnly = "secret";  // block-scoped
    console.log(localOnly);     // works here
  }
  // console.log(localOnly);   // Error — not visible out here
}
```

Debugging basics

Debugging is the process of finding and fixing problems in your code. Some go-to techniques:

- `console.log()` at key points to check what values actually are.
- Read the **error message** carefully — it usually tells you the line and the type of error.
- Use browser DevTools' **Sources** tab to set breakpoints and step through code line by line.

Common error types you'll meet as a beginner:

- **SyntaxError** — the code isn't written correctly (e.g. a missing bracket).
- **ReferenceError** — you're using a variable that doesn't exist / isn't in scope.
- **TypeError** — you're trying to do something invalid with a value's type (e.g. calling a number like a function).

Key Points

- `let` and `const` are block-scoped; `var` is function-scoped.
- Read error messages top to bottom — they point you to the problem.
- `console.log` liberally while learning; remove it once the bug is fixed.

Part 2 — Functions, Arrays & Objects

5. Functions

A function is a reusable block of code that performs a task. You define it once and can call it as many times as you like.

```
// Function declaration
function add(a, b) {
  return a + b;
}

console.log(add(2, 3));   // 5

// Function expression
const multiply = function (a, b) {
  return a * b;
};
```

Values passed into a function are called **parameters** (in the definition) or **arguments** (when calling it). `return` sends a value back out of the function — once `return` runs, the function stops.

Parameters can have default values, used when no argument is passed:

```
function greet(name = "friend") {
  return `Hello, ${name}!`;
}

greet();          // "Hello, friend!"
greet("Zoe");     // "Hello, Zoe!"
```

Key Points

- Functions help you avoid repeating code — write once, call many times.
- `return` gives back a value; without it, a function returns `undefined`.
- Give parameters default values to make functions more flexible.

6. Arrow Functions & Callbacks

Arrow functions are a shorter way to write function expressions, introduced in modern JavaScript (ES6).

```
// Regular function expression
const square = function (x) {
  return x * x;
};
```

```
// Arrow function — same thing
const squareArrow = (x) => {
  return x * x;
};

// Arrow function with an implicit return (no braces needed)
const squareShort = (x) => x * x;
```

A **callback** is simply a function passed into another function as an argument, to be run later. They're everywhere in JavaScript — timers, events, and array methods all use them.

```
// setTimeout takes a callback and a delay in milliseconds
setTimeout(() => {
  console.log("This runs after 1 second");
}, 1000);

// A button click handler is also a callback
button.addEventListener("click", () => {
  console.log("Button was clicked!");
});
```

Key Points

- Arrow functions `(params) => expression` are shorter, especially for simple one-line functions.
- A callback is just "a function you hand to another function to run later."
- You'll see callbacks constantly with events, timers, and array methods (Lesson 10).

7. Arrays

An array is an ordered list of values, stored in a single variable. Items are accessed by their **index**, starting at 0.

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits[0]);      // "apple"
console.log(fruits.length);  // 3

fruits.push("date");         // add to the end
fruits.pop();                 // remove from the end
fruits.unshift("kiwi");      // add to the start
fruits.shift();               // remove from the start
```

Some other handy array methods:

- `indexOf(value)` — find the position of a value (or `-1` if not found).
- `includes(value)` — check whether a value exists (returns `true`/`false`).

- `slice(start, end)` — copy out a portion, without changing the original array.
- `splice(start, count)` — remove or replace items, changing the original array.

You can loop over an array with a regular `for` loop, `for...of`, or `forEach`:

```
for (const fruit of fruits) {
  console.log(fruit);
}

fruits.forEach((fruit) => {
  console.log(fruit);
});
```

Key Points

- Array indexes start at **0**, not 1.
- `push`/`pop` work on the end; `unshift`/`shift` work on the start.
- `for...of` and `forEach` are the cleanest ways to loop over array values.

8. Objects

An object stores data as **key–value pairs** — useful for grouping related information together.

```
const student = {
  name: "Priya",
  age: 17,
  isEnrolled: true,
  greet: function () {
    return `Hi, I'm ${this.name}`;
  },
};

console.log(student.name);       // dot notation → "Priya"
console.log(student["age"]);    // bracket notation → 17
console.log(student.greet());    // "Hi, I'm Priya"
```

Use **dot notation** (`student.name`) when you know the exact key name. Use **bracket notation** (`student["name"]`) when the key is stored in a variable, or isn't a valid identifier.

A function stored inside an object is called a **method**. Inside a method, `this` refers to the object it belongs to.

You can add, update, or delete properties after creation:

```
student.age = 18;          // update
student.grade = "12th";    // add a new property
delete student.isEnrolled; // remove a property
```

Key Points

- Objects group related data using named keys, not numeric indexes.
- Dot notation for known keys; bracket notation for dynamic/variable keys.
- A function inside an object is a **method**; `this` refers to that object.

9. Destructuring & Spread

Destructuring is a shortcut for pulling values out of arrays or objects into their own variables.

```
// Array destructuring
const colors = ["red", "green", "blue"];
const [first, second] = colors;
console.log(first);   // "red"

// Object destructuring
const person = { name: "Sam", age: 22 };
const { name, age } = person;
console.log(name);    // "Sam"
```

The **spread operator** (`...`) expands an array or object out into individual elements — handy for copying or combining data without changing the original.

```
const nums1 = [1, 2, 3];
const nums2 = [...nums1, 4, 5];   // [1, 2, 3, 4, 5]

const base = { name: "Sam" };
const extended = { ...base, age: 22 };  // { name: "Sam", age: 22 }
```

The **rest parameter** looks similar (`...`) but works the opposite way — it gathers multiple arguments into a single array, usually in a function definition:

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}

sum(1, 2, 3, 4);   // 10
```

Key Points

- Destructuring unpacks values from arrays/objects into named variables.
- Spread (`...`) expands a collection out — great for copying and merging.
- Rest (`...`) does the opposite — it gathers arguments into an array.

10. Array Methods — map, filter & reduce

These three methods are the workhorses of working with arrays. Each one takes a callback function and returns a **new** array (or value) — none of them change the original array.

map — transform every item

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map((n) => n * 2);
console.log(doubled);   // [2, 4, 6, 8]
```

filter — keep items that pass a test

```
const numbers = [1, 2, 3, 4, 5, 6];
const evens = numbers.filter((n) => n % 2 === 0);
console.log(evens);     // [2, 4, 6]
```

reduce — combine everything into one value

```
const numbers = [1, 2, 3, 4];
const total = numbers.reduce((accumulator, n) => accumulator + n, 0);
console.log(total);     // 10
```

In `reduce`, the **accumulator** carries the running result from one call to the next, and the second argument (here `0`) sets its starting value.

Key Points

- `map` → same number of items, transformed.
- `filter` → fewer (or equal) items, only the ones that pass the test.
- `reduce` → collapses the array down to a single value.

11. ES Modules

As projects grow, it helps to split code across multiple files. ES Modules let one file **export** code so another file can **import** and use it.

```
// mathUtils.js
export function add(a, b) {
  return a + b;
}

export const PI = 3.14159;
```

```
// This is the default export — only one allowed per file
export default function greet(name) {
  return `Hello, ${name}`;
}

// main.js
import greet, { add, PI } from "./mathUtils.js";

console.log(add(2, 3));   // 5
console.log(PI);          // 3.14159
console.log(greet("Ana")); // "Hello, Ana"
```

To use modules in the browser, add `type="module"` to your script tag:

```
<script type="module" src="main.js"></script>
```

Key Points

- **Named exports** (`export function/const`) — import with matching `{ }` names.
- **Default export** — one per file, imported without curly braces.
- Modules keep code organized, reusable, and easier to maintain.